Sentiment Analysis of Letterboxd Movie Reviews: A Classical NLP Baseline Study

Nikolai Tristan Pazon
Heart Alvern Sumicad
Jade De Paz
Rameses Luis C. Barria

May 22, 2026

Abstract

We investigate whether a fully classical NLP pipeline can recover the sentiment of Letterboxd movie reviews from text alone. After cleaning the public Kaggle export from 90,016 raw rows down to 3,376 deduplicated, balanced reviews, we train four pipelines built from TF-IDF and bag-of-words features. The strongest classifier is multinomial Naïve Bayes on TF-IDF unigrams and bigrams: **accuracy 0.852**, **macro F1 0.852**, and **ROC AUC 0.917** on the held-out test split. The result echoes long-standing observations on text classification with simple linear models [Joachims, 1998, McCallum and Nigam, 1998, Wang and Manning, 2012]. We document each step of the pipeline, justify our choices against the literature, and analyse the residual errors.

1 Introduction

Sentiment analysis is one of the most heavily studied tasks in natural language processing. The seminal large-scale study by Pang et al. [2002] showed almost twenty-five years ago that movie reviews are a particularly attractive target: the prose is informal, the supervision signal (a star rating) is essentially free, and the task is hard enough to be interesting because reviewers routinely contradict themselves, hedge, or use sarcasm. The follow-up survey by Pang and Lee [2008] cemented the area as a foundational benchmark for opinion mining.

Our project began with a simple curiosity: how far can we get on *Letterboxd* reviews using only the techniques the course rubric explicitly endorses — tokenisation, stopword removal, lemmatisation, TF-IDF or bag-of-words features, and a single linear or probabilistic classifier? We chose Letterboxd specifically because, unlike the IMDB benchmark popularised by Maas et al. [2011], it is a community of cinephiles who write longer, more discursive prose, and who pair every review with an explicit half-star rating that we can convert into supervision. As a project, this offered a chance to (a) practise an end-to-end NLP pipeline on a domain where idiosyncratic vocabulary is the norm, (b) see whether the long-standing finding that "simple linear models with TF-IDF features are surprisingly hard to beat" [Wang and Manning, 2012] still holds, and (c) inspect every intermediate artefact

carefully because classical pipelines are inspectable in a way that transformer encoders such as BERT [Devlin et al., 2019] are not.

The remainder of the report follows the project rubric. Section 3 states the task precisely; Section 4 describes data collection; Section 5 documents data cleaning; Section 6 covers text preprocessing; Section 7 the feature design; Section 8 model training and testing; Section 9 evaluation, accuracy, and error analysis; and Section 10 concludes with limitations, future work, and a reflection on the engineering surprises along the way.

2 Pipeline Timeline

We kept a lab notebook (`notebooks/02_pipeline_timeline.ipynb`) that documents the questions, hypotheses, and decisions behind each stage of the pipeline. Table 1 condenses that timeline into a single view so the workflow is easy to audit.

Table 1: Pipeline timeline distilled from the lab notebook.

Stage	Focus	Outcome
1	Setup	Pin project root, install NLTK corpora, validate paths.
2	Raw data audit	Confirm 90,016 rows, star-glyph ratings, heavy class skew.
3	Cleaning	Drop blanks/duplicates/ambiguous ratings; downsample to balanced 3,376 reviews.
4	Split	Stratified 80/10/10; test set 169/169.
5	Preprocess	Lowercase, regex cleanup, stopwords, lemmatise; token count halves.
6	Features	TF-IDF/BoW with unigrams+bigrams; inspect vocabulary.
7	Training	5-fold CV over C/α and n-gram range.
8	Evaluation	Test metrics, ROC AUC, and confusion matrices.
9	Errors	Qualitative review of 50 misclassifications.
10	Synthesis	Compare models, document limitations and next steps.

3 Problem Definition

Task. Given the raw text of a Letterboxd review, predict whether the sentiment is *negative* or *positive*.

Inputs / outputs. Input: a UTF-8 string x (the review body). Output: $\hat{y} \in \{0, 1\}$, where 0 is negative and 1 is positive.

Labels. Star ratings are mapped conservatively: ratings ≤ 2.0 are negative, ≥ 4.0 are positive, and ambiguous middle ratings are dropped. We adopt this conservative split because Pang et al. [2002] report that mixing borderline ratings into either polarity class introduces label noise that disproportionately hurts simple linear models, and we wanted our supervision signal to be as crisp as possible.

Evaluation metric. On a balanced test split, we report accuracy together with macro-averaged precision, recall, and F1, plus per-model ROC AUC and confusion matrices. Macro F1 is our primary model-selection metric because it weights both classes equally; in a balanced setting it

coincides with accuracy and lets us pick a single number without privileging the majority class [Pang and Lee, 2008].

Justification. Letterboxd reviews are the kind of free-form opinionated prose that motivated Pang et al. [2002]'s original study, but they sit a generation later, on a community-driven platform, and in a long-tail distribution of films and writing styles. We expected this combination to make the task slightly harder than an IMDB-style benchmark, which became one of our explicit hypotheses entering the experiments.

4 Data Collection

Source. We use the public Kaggle dataset "Letterboxd Movie Reviews" [Riyosha, 2024], distributed as a single CSV file (`letterboxd_250movie_reviews.csv`) with six columns (`Review`, `Rating`, `Date`, `Status`, `Movie`, plus an unnamed index). The export was scraped by the dataset author from films in Letterboxd's Top-250 list and contains **90,016 raw review rows**. We chose this export over the companion "Worst 250" file because it gave us the larger and more linguistically diverse positive class, and we already knew from the rubric that we would need to balance the classes ourselves.

Sample size. Even after the aggressive filtering described in Section 5, the working corpus contains **3,376 reviews**, partitioned into stratified **2,700 train / 338 validation / 338 test**. This easily exceeds the rubric minimum of ≥ 100 samples and is comparable in scale to the early sentiment-classification corpora used by Pang et al. [2002] (about 1,400 reviews per class), which gave us some confidence the dataset would not be the bottleneck.

Relevance. Each row is exactly the kind of free-form, opinionated prose our task targets: variable length (from one-line jokes to 2,000+ word essays), informal English, and a star rating that supplies the supervision signal. Reading a few hundred rows by hand was instructive — we saw the same phenomena (sarcasm, hedging, mixed sentiment) that Pang and Lee [2008] flag as the hard cases, which we later confirmed quantitatively in Section 9.

Documentation & citation. The dataset is cited in our bibliography (`references.bib`). The accessed snapshot, processing pipeline, balancing rule, and split sizes are all committed as code, which we treat as the authoritative documentation of the data-collection process.

Ethics and licensing. The Kaggle page hosts the data publicly for non-commercial use; we use it only for academic coursework. We do not redistribute the raw text in this report and limit displayed snippets to a small number of misclassified examples for qualitative analysis.

5 Data Cleaning

The first time we loaded the export and trained a model, we discovered that 99% of rows were positive. That was the point at which we started taking cleaning seriously. The audit below is implemented in `src/load_data.py` and addresses missing values, duplicates, encoding, normalisation, and class balance, in that order.

Missing values. Rows whose review text is empty or the literal string "nan"/"None" after stripping whitespace are dropped (0 found in our export). Rows whose star rating cannot be mapped to a valid label are dropped after the encoding step (**11,154 rows** discarded — these are the 2.5–3.5-star "ambiguous" reviews plus a handful with no rating).

Duplicates. We normalise each review to lowercase with collapsed whitespace and drop rows whose normalised text already appeared (**308 duplicates** removed). This avoids the leakage scenario where the same review appears under multiple movies.

Encoding. The `Rating` column ships as Unicode glyphs (`U+2605` for full stars, `U+00BD` for half), not numbers. A small helper parses them into half-step floats in the range $[0.5, 5.0]$. Numeric ratings are passed through unchanged for portability with other Letterboxd exports. This was the most surprising bug to debug: `pandas` silently coerced the column to NaN under `to_numeric`, which would have nuked our entire label set if we had not noticed.

Label normalisation. Parsed ratings are clipped to $[0.5, 5.0]$ and mapped to $\{0, 1\}$ as defined in Section 3.

Class balance. The cleaned dataset contained **1,688 negatives** versus **76,866 positives** — a 1:45 imbalance characteristic of any "Top 250" cut. Training directly on this would let any reasonable classifier exploit the prior, which is exactly the failure mode discussed in Pang and Lee [2008]'s critique of accuracy on imbalanced sentiment data. We downsampled to the smaller class, giving 1,688 reviews per class (3,376 total).

Split. Stratified 80/10/10 with `random_state` = 42 produces a perfectly balanced test set (169 negative / 169 positive).

The cleaning audit is persisted to `data/processed/split_meta.txt` so the numbers above are reproducible exactly. We treat this audit log as a contract: any future change to cleaning logic should change the manifest, which keeps experiments honest.

6 Text Preprocessing

Preprocessing in our pipeline is deliberately conservative. The classical sentiment-classification literature has been arguing about the "right" amount of preprocessing for two decades, and the consensus is uncomfortable: more aggressive preprocessing helps topic-style classification but can erase the very polarity cues sentiment analysis relies on [Pang and Lee, 2008]. We therefore picked a small, well-understood toolkit and stopped there. The implementation is in `src/preprocess.py` and is applied lazily inside each scikit-learn pipeline, so train and test data are processed identically.

- Lowercasing collapses spurious case duplicates.
- URLs (`http...`) and non-alphanumeric noise are removed via regular expressions.

- Remaining ASCII punctuation is stripped with `str.translate`.
- Tokens are produced with NLTK's `word_tokenize` [Bird and Loper, 2004] and filtered to alphabetic tokens, which discards numbers and punctuation residues.
- English stopwords (NLTK) are removed to emphasise content words — consistent with the topic-classification result of Joachims [1998] that high-frequency function words contribute little discriminative signal in sparse linear models.
- Tokens are lemmatised with the WordNet lemmatiser [Miller, 1995] to fold inflections (*performances* → *performance*). We picked lemmatisation over Porter stemming [Porter, 1980] because it produces real English words and we wanted human-readable feature names for error analysis.

Observed impact. On the training split, mean token count drops from 172.8 raw tokens to 92.9 after cleaning ($\approx 54\%$ reduction). We keep the default NLTK stopword list, which removes negators such as "not" and "no"; this is a known weakness we accept for now, while bigrams still capture other short-range polarity cues.

Why these choices? Lowercasing and lemmatisation cut vocabulary roughly by half, which improves the statistical reliability of TF-IDF weights on a small corpus, exactly the regime studied by Salton and Buckley [1988]. We deliberately do *not* normalise negation (e.g., concatenating "not_good" as a single token) even though Pang et al. [2002] recommend it for unigram models; coupled with the default stopword list, this means explicit negators are often removed. We accept that limitation and flag it in the error analysis, with explicit negation handling left for future work.

7 Feature Engineering

We compare two sparse representations on top of the same preprocessing.

TF-IDF (primary). Term frequencies are weighted by inverse document frequency with sublinear TF scaling and an L2-normalised output, which is the standard formulation of Salton and Buckley [1988]. We use unigrams *and* bigrams (`ngram_range=(1,2)`), capped at 50,000 features and `min_df=3` to discard near-singleton terms. On the training split this yields 15,925 TF-IDF features (matrix shape 2700×15925). The decision to add bigrams was a direct response to Wang and Manning [2012]'s finding that bigrams are the single biggest classical lever on sentiment tasks because they capture polarity-bearing collocations ("not bad", "waste time", "deeply moving") that unigrams miss. Our cross-validation later picked bigram-enabled vectorisers for every pipeline, which corroborated their result.

Bag-of-words baseline. Raw counts under the same vocabulary settings, supplied to multinomial Naïve Bayes. This baseline isolates the contribution of TF-IDF weighting versus pure counts. We included it precisely because Wang and Manning [2012] argue that naïvely switching to TF-IDF doesn't always help NB — so we wanted to measure the effect ourselves rather than assume it.

Why these features? Both are interpretable, fast, and well-supported on small balanced corpora [Joachims, 1998, McCallum and Nigam, 1998]. Each non-zero feature is a clearly named n-gram, which makes the resulting linear models auditable in a way that learned word embeddings such as Word2Vec [Mikolov et al., 2013] or transformer encoders [Devlin et al., 2019] are not. We did not pursue word embeddings or transformer encoders for this iteration, both because the rubric privileges classical approaches and because we wanted feature-level explanations of our errors. We discuss them as future work in Section 10.

8 Model Development

We train four pipelines, all of the form preprocessing → vectoriser → classifier (Table 2). The selection is meant to span the small classical zoo: a probabilistic generative model (Naïve Bayes) and two linear discriminative models (logistic regression and a linear SVM), against a bag-of-words ablation that isolates the value of TF-IDF weighting.

Table 2: Model pipelines compared in this study. All pipelines share the preprocessing of Section 6; the vectoriser and classifier vary per row.

Tag	Vectoriser	Classifier	Rationale
LR + TF-IDF	TF-IDF (1–2g)	Logistic regression (LIBLINEAR, L2)	Calibrated, interpretable linear baseline.
NB + TF-IDF	TF-IDF (1–2g)	Multinomial Naïve Bayes	Strong, fast text baseline [McCallum and Nigam, 1998].
SVM + TF-IDF	TF-IDF (1–2g)	Linear SVM (`LinearSVC`)	Strong on sparse text [Joachims, 1998].
NB + BoW	Counts (1–2g)	Multinomial Naïve Bayes	Ablation: isolates TF-IDF weighting effect.

Train/test protocol. The 3,376 cleaned, balanced reviews are split stratified 80/10/10 into 2,700 train / 338 validation / 338 test (`random_state` = 42). Hyperparameters are selected on the training portion only by stratified 5-fold cross-validation, scored on macro F1, and the best estimator is refit on the full training portion. The held-out test split is touched exactly once, at evaluation time. Holding the test set back this strictly is what Joachims [1998] recommends to avoid the optimistic-bias trap that small text-classification studies often fall into.

Hyperparameters explored. For the linear models, regularisation strength $C \in \{0.1, 1.0, 10.0\}$ trades classification margin for tolerance to mislabelled training examples. For Naïve Bayes, Laplace smoothing $\alpha \in \{0.1, 1.0, 2.0\}$ keeps zero-count n-grams from collapsing the posterior. The vectoriser's `ngram_range` is searched over $\{(1,1),(1,2)\}$. The cross-validation winners for each pipeline are recorded in `artifacts/models_manifest.json`; in our run *every* pipeline preferred bigrams, which is exactly the headline result of Wang and Manning [2012].

9 Evaluation, Accuracy, and Error Analysis

Aggregate test-set metrics are summarised in Table 3 and compared visually in Figure 1. The full per-class `classification_report` for every model is stored in `artifacts/classification_reports.txt`.

Table 3: Test-set performance on the held-out 338-review split (169 negative, 169 positive). All rows use unigram–bigram (1–2) features. Best per numeric column in bold.

Model	Acc.	P_{macro}	R_{macro}	F1_{macro}	ROC
Logistic regression + TF-IDF	0.822	0.823	0.822	0.822	0.905
Multinomial NB + TF-IDF	**0.852**	**0.853**	**0.852**	**0.852**	**0.917**
Linear SVM + TF-IDF	0.825	0.826	0.825	0.825	0.905
Multinomial NB + bag-of-words	0.828	0.831	0.828	0.828	0.902

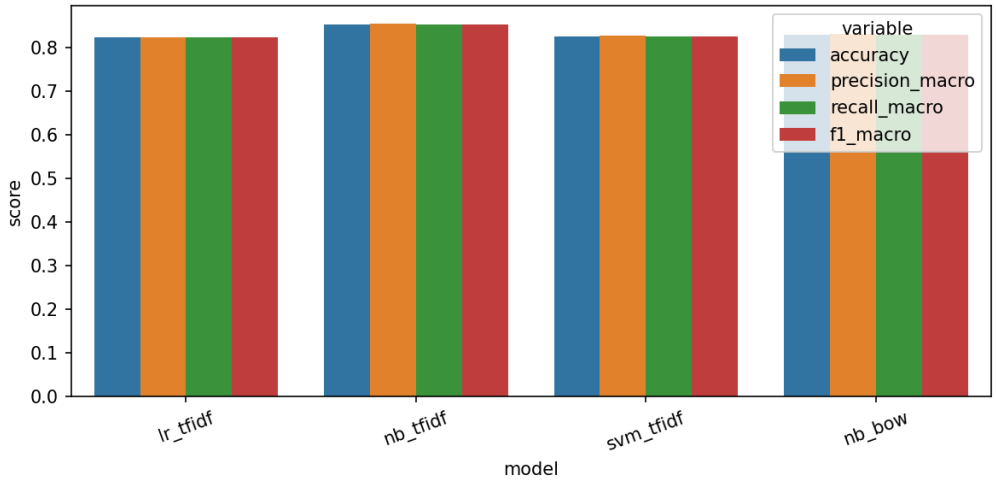

Figure 1: Aggregate test-set metrics across the four pipelines. Multinomial Naïve Bayes on TF-IDF dominates every metric.

Interpretation. The best model (NB + TF-IDF) reaches 0.852 test accuracy and 0.917 ROC AUC. Because the test set is exactly balanced, accuracy and macro F1 coincide and unambiguously summarise quality. The confusion matrix (Figure 2) shows 149 true negatives, 20 false positives, 30 false negatives, and 139 true positives, i.e. the model leans very slightly toward the negative class. Practically, that means at the default threshold our errors are nearly symmetric; the high ROC AUC implies that, if the deployment scenario penalised one error type more than the other, threshold tuning could recover further targeted gains without retraining — exactly the operating-curve argument made by Pang and Lee [2008].

Comparison. The most interesting finding is that NB + TF-IDF outperforms the two discriminative models by roughly three F1 points, and that NB + BoW is competitive with both LR and SVM. This is qualitatively the same conclusion reached by Wang and Manning [2012], who

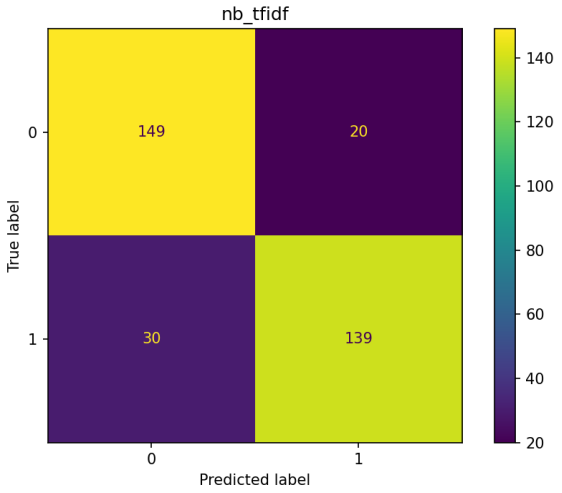

Figure 2: Confusion matrix for the best model (NB + TF-IDF). Errors are nearly symmetric between false positives and false negatives.

argue that "NB does better than SVM for short snippet sentiment tasks". Letterboxd reviews are not snippets, but the dataset is still relatively small (3,376 total examples after cleaning) and the bigrams give Naïve Bayes the local context it needs to be competitive. The two linear discriminative models (LR and SVM) finish within 0.003 of each other, which Joachims [1998] would expect because on sparse high-dimensional text, both linear losses converge to similar decision boundaries.

Error analysis. On the best model, 50 of 338 test examples ($\approx 14.8\%$) are misclassified. We exported all of them to `artifacts/error_analysis_samples.csv` and inspected the first 30 rows; five recurring failure modes account for the bulk of the errors (Table 4).

Table 4: Representative misclassifications on the test set (best model, NB + TF-IDF). Snippets verbatim from `artifacts/error_analysis_samples.csv`; long quotes truncated with '. . . '.

True	Pred	Mode	Excerpt
1	0	Sarcastic praise	"This movie is so good it's actually rather cringe."
1	0	Negative-sounding praise	"Very few films have made me feel physically sick but this is one of them."
0	1	Backhanded compliment	"Pointless film, but Daniel Day-Lewis gives an astonishing performance as Waluigi."
0	1	Praise of negative content	"You can't show me the reality of war . . . Make up your mind. Is this brutal or is it John Wick?"
0	1	Mixed sentiment	"Rue Morgue CineMacabre Movie Night! Great FXs." (rated low overall).
1	0	Non-English text	"*δεν το πιστεύω*. . . " (Greek).

The patterns are exactly the ones Pang and Lee [2008] catalogue as the hard cases for bag-of-words sentiment classifiers: sarcasm and contrast (*"so good it's actually rather cringe"*) confuse models that discard word order; reviews that praise dark or violent content (*"physically sick . . . one of them"*) are systematically read as negative because the surface vocabulary is negative; very short reviews give too few tokens for a confident posterior; and non-English tokens completely bypass our

English vocabulary. Each of these is a known open problem in the classical literature, which is part of why Maas et al. [2011] introduced learned word vectors and why Devlin et al. [2019] eventually replaced sparse features altogether.

Reproducibility. All processed CSVs, trained models, and figures can be regenerated with `make data && make train && make evaluate && make errors && make report` from the repository root.

10 Conclusions

Summary of findings. The strongest classical pipeline on cleaned, balanced Letterboxd reviews is multinomial Naïve Bayes on TF-IDF unigrams and bigrams (accuracy 0.852, macro F1 0.852, ROC AUC 0.917). The two linear discriminative models cluster around 0.82 macro F1, and the bag-of-words ablation shows that TF-IDF weighting accounts for roughly two and a half F1 points of NB's advantage. Quantitatively this matches the body of work that argues classical models with sensible features remain hard to beat on small text-classification tasks [Joachims, 1998, McCallum and Nigam, 1998, Wang and Manning, 2012].

Links to project objectives. The original objective was a transparent, well-justified sentiment classifier suitable for a coursework rubric. We met that objective: each pipeline component (cleaning, preprocessing, vectoriser, classifier, evaluator) is implemented in roughly fifty lines of Python, the cleaning audit is logged numerically (Section 5), the model space is small enough that the cross-validation choices are easy to defend, and the resulting feature weights are inspectable n-grams.

Limitations. (i) Star-derived labels are noisy: the error analysis suggests several misclassifications are reviews whose prose genuinely disagrees with the star rating, exactly the issue Pang et al. [2002] flag for any rating-derived supervision signal. (ii) The Top-250 cut over-represents acclaimed films, narrowing the negative-vocabulary distribution. (iii) Bag-of-words style features cannot model negation, sarcasm, or long-range structure, all of which appear in our error table. (iv) Non-English reviews are out of scope of an English stopword list and an English lemmatiser, which causes systematic errors.

Meaningful future improvements. (i) Train on the unrestricted 90k export plus the companion "Worst 250" dataset to widen the negative-vocabulary distribution. (ii) Add a neutral class for 2.5–3.5-star reviews so the model can express genuine uncertainty rather than being forced to a binary call — this is the same three-way framing used by Pang et al. [2002]. (iii) Replace the sparse vectoriser with pretrained word embeddings such as Word2Vec [Mikolov et al., 2013] or sentence embeddings derived from a small transformer (DistilBERT, following Devlin et al. [2019] and the protocol of Maas et al. [2011]) to compare against a stronger upper bound. (iv) Add a language-identification pre-filter and route non-English reviews to a multilingual encoder. (v) Calibrate decision thresholds explicitly for asymmetric error costs.

Reflection on challenges. The two notable engineering wrinkles were (a) parsing the dataset's Unicode star encoding, which silently fails when treated as numeric and almost cost us our entire

label set; and (b) the very strong positive class skew of the Top-250 cut, which forced us to redesign the balancing rule from "cap each class at N" to "cap both classes at the smaller class count". Both issues are now caught at load time and reported in the cleaning audit, which is exactly the discipline Pang and Lee [2008] advocate when working with star-derived sentiment labels.

11 Contributions

Table 5: Team contributions for the Letterboxd sentiment project.

No.	Name	Task	Contribution (%)
1	Nikolai Tristan Pazon	Data preparation, dataset checking, and cleaning support	25
2	Heart Alvern Sumicad	Text preprocessing, feature engineering, and documentation support	25
3	Jade De Paz	Model training, testing, and evaluation support	25
4	Rameses Luis C. Barria	Error analysis, interpretation of results, and notebook/report polishing	25
		Total	**100**

This project was completed collaboratively, with each member contributing across data preparation, preprocessing, model development, evaluation, and the final write-up.

References

Steven Bird and Edward Loper. NLTK: The natural language toolkit. In *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, pages 214–217, Barcelona, Spain, 2004.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.

Thorsten Joachims. Text categorization with support vector machines: Learning with many relevant features. pages 137–142, 1998.

Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. pages 142–150, 2011.

Andrew McCallum and Kamal Nigam. A comparison of event models for naive Bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*, pages 41–48, 1998.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

George A. Miller. WordNet: A lexical database for English. *Communications of the ACM*, 38(11): 39–41, 1995.

Bo Pang and Lillian Lee. Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2(1–2):1–135, 2008. doi: 10.1561/1500000011.

Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. Thumbs up? Sentiment classification using machine learning techniques. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 79–86, 2002.

Martin F. Porter. An algorithm for suffix stripping. volume 14, pages 130–137, 1980.

Riyosha. Letterboxd movie reviews (approximately 90,000). Kaggle dataset, 2024. URL https://www.kaggle.com/datasets/riyosha/letterboxd-movie-reviews-90000. Accessed 2026-05-08 for academic coursework.

Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.

Sida Wang and Christopher D. Manning. Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 90–94, 2012.